Projeto de Implementação Desktop – Sistemas Distribuídos 2026/2

Objetivo do projeto: Trabalhar os conceitos da arquitetura cliente servidor em camadas, exclusão mútua distribuída, replicação e consistência distribuída. A linguagem de programação a ser utilizada é o Java, e toda a comunicação entre elementos do sistema deve ser realizada via sockets TCP.

Formato: O uso de IA para geração de código é livre. O trabalho deve ser realizado em dupla (escolha livre). Não serão aceitos trabalhos individuais ou em trio.

Arquitetura do sistema: 2 Clientes, 1 Balanceador de Carga, 3 Servidores de Aplicação e 2 Servidores de Dados.

Clientes: Loop infinito sorteando um número entre 1 e 1.000, que será enviado a um Servidor de Aplicação (escolhido pelo Balanceador) para verificação se o valor sorteado é ou não primo e se já está inserido no arquivo dos servidores de dados. Após um envio de requisição, o cliente dorme um tempo aleatório entre 50 e 200 ms. Assim, os clientes ficam em loop infinito enviando requisições e dormindo.

Balanceador: Distribuir as requisições entre os 3 servidores de aplicação, com uma política de balanceamento de carga a ser definida pela dupla.

Servidores de Aplicação: Para cada requisição recebida pelo Balanceador, uma nova thread worker é aberta. Essa thread worker deve verificar se o número recebido é ou não primo (por meio de qq algoritmo de primalidade a critério da dupla) , enviar esse número a um servidor de dados e aguardar a resposta desse servidor de dados sobre a existência ou não desse valor nos arquivos locais dos mesmos. Com base nessa resposta do servidor de dados, a thread worker deve imprimir na tela a mensagem “O número X é (não é) primo, não está presente no arquivo, e foi inserido na linha Y” ou “O número X é (não é) primo, e já se encontra no arquivo na linha Z”. Atentar que a thread worker deve fazer a verificação da primalidade primeiro e permanecer aberta aguardando a resposta do servidor de dados sobre a presença ou não do número no arquivo. Assim, somente após a impressão do resultado da primalidade e da existência ou não do valor no arquivo do servidor de dados a thread worker deve ser finalizada. A definição de qual dos servidores de dados irá atender a thread worker aberta é aleatória (sorteio), sem necessidade de uma política de balanceamento de carga específica.

Servidores de Dados: Cada servidor de dados terá um arquivo local associado. Para cada valor recebido de uma thread worker dos servidores de aplicação, o servidor de dados deve realizar os seguintes passos:

i) dormir um tempo aleatório entre 100 e 300ms e;

ii) verificar se esse valor já existe ou não no arquivo. Caso exista, o servidor de dados não irá escrever no arquivo, e deverá responder à thread worker que o número já existe, conforme explicado anteriormente. Caso não exista, o servidor de dados cria uma nova linha no seu arquivo local (a 1ª inserção deve ser feita na 1ª linha do arquivo) para inserir o novo valor no arquivo e responder à thread worker dessa inserção, conforme também já explicado anteriormente.

iii) Realizar a consistência entre os arquivos locais de cada servidor de dados, caso haja um procedimento de escrita. O mecanismo de consistência a ser utilizado e como o mesmo será implementado deve ser definido pela dupla. Entretanto, deve ser garantido que após uma escrita (nova linha) em um dos arquivos a consistência seja disparada, e que nenhuma nova requisição das threads seja atendida antes do término da consistência. Desta forma, ambos os arquivos possuirão os mesmos valores de números que passaram pelo teste de primalidade a cada nova escrita e ao término da execução do sistema.

Restrições do sistema:

- Vcs devem implementar os elementos descritos acima como objetos executáveis (IP localhost e porta a definir) independentes. A ideia é simularmos que os clientes estão em uma máquina cada um, cada servidor de aplicação em outras 3 máquinas e cada servidor de dados em outras 2 máquinas. Como vcs irão implementar tudo em uma única máquina na casa de vcs, o IP localhost e portas diferentes serão necessários para simularmos esse ambiente distribuído.

- Nenhuma requisição dos clientes pode ser perdida durante o procedimento de consistência de arquivos dos servidores de dados. Para verificar tal situação, vcs devem gerar um arquivo de log nos clientes onde a cada linha será escrito o novo valor sorteado. Desta forma, ao final da execução todos os números distintos gerados pelos clientes devem estar em seus respectivos arquivos de log e nos arquivos dos servidores de dados. Atentar que valores eventualmente repetidos sorteados pelos clientes estarão nesses logs, mas apenas um registro desse valor estará nos arquivos dos servidores de dados.

Entregas do projeto:

28/10 até às 23:59 – Entrega no Teams do código fonte da solução (todos os programas e elementos em Java que compõe a solução);

Aula de 29/10 – Apresentação da solução desenvolvida em aula presencial e execução do sistema (totalizando aproximadamente 20 minutos cada dupla). Nessa apresentação vocês explicarão o código fonte do projeto, além de abordarem os seguintes pontos:

- Qual política de balanceamento de carga foi utilizada no Balaceador, e a justificativa para tal;

- Qual o mecanismo de consistência foi utilizado nos Servidores de Dados, como ela foi implementada e a justificativa para tal escolha.

OBS: A apresentação da solução é livre. Vcs podem usar slides, diagramas, prints de tela, imagens, etc., ou seja, o que acharem mais conveniente para explicar a solução desenvolvida.

Pontuação do trabalho (total 10 pontos):

0-4 pontos: corretude e eficiência geral do sistema (atendimento do enunciado, arquitetura do sistema e eficiência do código, e execução sem erros);

0-1 ponto: Política de balanceamento de carga; (Explicação, justificativa, eficiência e corretude)

0-3 pontos: Mecanismo de consistência entre os arquivos; (Explicação, justificativa, eficiência e corretude)

0-2 pontos: Apresentação em sala (Clareza da explicação da solução).

Qq dúvida ou esclarecimento adicional sobre o trabalho, é só chamar no chat.

Bons estudos!